CO 2 – ASSESMENT TOOL 2

JAVA STRING HANDLING AND COLLECTION FRAMEWORK

NAME: P.V.N. ANISH BABU

REG NO: 192472329

COURSE CODE: CSA0924

COURSE NAME: JAVA PROGRAMMING

SLOT: D

Evaluation of Java Generic Wildcards

AIM

To evaluate List<?>, List<Object>, List<? extends Number>, and List<? super Integer> and determine what can be safely read and what can be safely added using Java Generics.

PSEUDO CODE

START

Create four lists:

1. List<?>
2. List<Object>
3. List<? extends Number>
4. List<? super Integer>

For each list:

Check what type can be read safely.

Check what elements can be added safely.

Display the results.

STOP

JAVA CODE

```
from typing import List, Any, Union

print("Python Equivalent Concepts")
print("--------------------------")

# Simulating List<?> in Python (List[Any])
string_list: List[str] = ["Python"]

# A list that can hold anything (similar to List<?>)
list1: List[Any] = string_list

# Reading is safe, as in Java
obj1: Any = list1[0]
print(f"List[Any] (from List[str]) read: {obj1}")

# In Python, you *can* add different types to a dynamically typed list,
list1.append(100) # This would be a type error if list1 was only List[str]
print(f"List[Any] after adding int: {list1}")

# Simulating List<Object> in Python (List[Any])
# Again, Python's default list behavior is like List<Object>.
list2: List[Any] = []
list2.append("Python")
list2.append(100)
list2.append(45.5)
```

```
obj2: Any = list2[0]
print(f"List[Any] (heterogeneous) read: {obj2}")
print(f"Full List[Any]: {list2}")
# Simulating List<? extends Number> in Python (List[Union[int, float, complex]])

int_list: List[int] = [10, 20]

list3: List[Union[int, float, complex]] = int_list
num: Union[int, float, complex] = list3[0]
print(f"List[Union[int, float, complex]] (from List[int]) read: {num}")

# In Python, you can append different numeric types to list3 if it's declared as List[Union[...]]
# The restriction 'can only read out numbers' is enforced by how you *use* the elements.
list3.append(30.5) # Valid
print(f"List[Union[int, float, complex]] after adding float: {list3}")

# Simulating List<? super Integer> in Python (List[Union[int, Any]])
number_list: List[Union[int, float]] = [] # Can hold int or float

# A list that can accept integers (List[Any] works or a more specific Union)
list4: List[Any] = number_list # Assigning a more specific list to a more general one
list4.append(10) # Can add int
list4.append(20) # Can add int
list4.append(3.14)
obj4: Any = list4[0]
print(f"List[Any] (acting as supertype for int) read: {obj4}")
print(f"Full List[Any] after adding int and float: {list4}")
```

OUTPUT

Python Equivalent Concepts

List [Any] (from List[str]) read: Python

List [Any] after adding int: ['Python', 100]

List [Any] (heterogeneous) read: Python

Full List [Any]: ['Python', 100, 45.5]

List [Union [int, float, complex]] (from List[int]) read: 10

List [Union [int, float, complex]] after adding float: [10, 20, 30.5]

List [Any] (acting as supertype for int) read: 10

Full List [Any] after adding int and float: [10, 20, 3.14]

Professional Analysis

Type	**Safe to Read**	**Safe to Add**
List<?>	Object	Only null
List<Object>	Object	Any object
List<? extends Number>	Number	Only null
List<? super Integer>	Object	Integer and subclasses

Summary Table

List Type	**Read Type**	**Add Allowed**
List<?>	Object	null only
List<Object>	Object	Any object
List<? extends Number>	Number	null only
List<? super Integer>	Object	Integer
List<Integer>	Integer	Integer

CONCLUSION

- List<?> is read-only except for null.
- List<Object> allows reading and writing any object.
- List<? extends Number> acts as a producer and supports safe reading as Number.
- List<? super Integer> acts as a consumer and safely accepts Integer objects.

This behavior follows the **PECS (Producer Extends, Consumer Super)** principle in Java generics.